

GPU-Accelerated LIF Spiking Neuron Networks Simulator

Your Name

Student ID

January 11, 2026

1 Introduction

The Project is done to implement more compact and lightweight SNN simulator compared to the ones already available.

The neuron implementation is numerical instead of analog. Firstly, the neuron is calculated, then the value being compared to the threshold voltage, and resulting a spike. The refractory phase will start for the spiked neuron. Lastly, the synaptic updates are being calculated as exponential decay per timestep.

2 Neuron Model

The discrete-time implementation of the Leaky Integrate-and-Fire (LIF) model as discussed by Stan and Rhodes [1] allows for efficient sequence modeling in SNNs.

The parameters in the equations are:

$u[t]$ Membrane voltage (potential) at time step t .

β Membrane decay factor.

$s[t]$ Spike indicator.

θ Firing threshold voltage.

$i[t]$ Input current.

The membrane potential update is being done with the equation 1.

The Reset mechanism is given in equation 2 as soft reset. Instead of resetting to a default value, the membrane voltage is being calculated directly. This prevent warp divergence if the hard reset chosen, with condition structure will be introduced.

Lastly, spike generation is being calculated with equation 3. The spike is being decided with comparison between membrane voltage and threshold.

$$u[t] = \beta u[t - 1] + (1 - \beta) i[t] \quad (1)$$

$$u[t] \leftarrow u[t] - s[t - 1] \theta \quad (2)$$

$$s[t] = \Theta(u[t] - \theta) = \begin{cases} 1, & \text{if } u[t] > \theta \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

where the decay factor β is defined by the membrane time constant τ and simulation time step Δt :

$$\beta = \exp\left(-\frac{\Delta t}{\tau}\right) \quad (4)$$

3 Synapse Model

The synapse model is decided as current-based exponential synapse model. The equation of the exponential synapse model is given in the equation 5.

Parameters are given for synapse model:

- α Synaptic decay factor ($e^{-\Delta t/\tau_s}$).
- w_{ji} Weight from neuron j to i .
- $s_j[t]$ Spike indicator from presynaptic neuron j .
- $g_i[t]$ Postsynaptic current state (decaying memory).
- $u_i[t]$ Membrane potential of neuron i .
- β Membrane decay factor ($e^{-\Delta t/\tau_m}$).
- θ Spike threshold.

$$g_i[t] = \alpha g_i[t-1] + \sum_j w_{ji} s_j[n] \quad (5)$$

Connecting this synapse model with the neuron model is resulted this equation 6. The input current is changed with the Postsynaptic current state, which is calculated in equation 5.

$$u_i[t] = \beta u_i[t-1] + (1 - \beta)g_i[t] \quad (6)$$

Current implementation of the model is dense. The sparsity can be added later.

4 Implementation

4.1 CPU Model

The CPU implementation is being done with the given neuron and synapse model definitions in previous chapters. It is written in C. The main time loop given in the figure 1.

To check the correctness of the C model, Pytorch version is also implemented with the same model definitions. The comparison test done. Firstly, the poisson (input) spikes are saved while running the pytorch version. Then those spikes are imported to the C version. This way the model expected to be run exactly the same. Meanwhile membrane potentials ($u[t]$) and spikes ($s[t]$) are being saved in both versions. At the end those saved values are being compared with a script, to check the correctness of the C implementation. Comparison output is given in the figure 2. As we can see from the given figure, the difference is pretty close to zero, with a little error margin because of the difference in float calculations.

```

/* Time loop */
for (int t = 0; t < T; ++t) {

    /* Synapse update */
    for (int i = 0; i < N; ++i) {
        float input_current = 0.0f;
        input_current += ext_spikes[t*N + i] * ext_weight;
        for (int j = 0; j < N; ++j) {
            input_current += Wflat[i*N + j] * s_prev[j];
        }
        g[i] = alpha * g[i] + input_current;
    }
    /* Neuron Update */
    for (int i = 0; i < N; ++i) {
        u[i] -= s_prev[i] * theta;
        u[i] = beta * u[i] + (1.0f - beta) * g[i];
        s[i] = (u[i] > theta);
    }
    /* Copy spikes */
    for (int i = 0; i < N; ++i) {
        s_prev[i] = s[i];
    }
}

```

Figure 1: Time loop of the C implementation

```

emre@emre-ABRA-A5-V16-6: ~/AllProjects/GPU-Accelerate...
t=17, i=5, u_py=0.273662, u_c=0.273662, diff=-2.980232e-07
t=17, i=6, u_py=0.026556, u_c=0.026556, diff=-9.313226e-09
t=17, i=7, u_py=0.013999, u_c=0.013999, diff=-9.313226e-10
t=18, i=0, u_py=0.325653, u_c=0.325653, diff=-1.490116e-07
t=18, i=1, u_py=0.024347, u_c=0.024347, diff=-3.725290e-09
t=18, i=2, u_py=0.009585, u_c=0.009585, diff=-9.313226e-10
t=18, i=3, u_py=0.018380, u_c=0.018380, diff=3.725290e-09
t=18, i=4, u_py=0.019884, u_c=0.019884, diff=1.862645e-09
t=18, i=5, u_py=0.298536, u_c=0.298536, diff=-2.682209e-07
t=18, i=6, u_py=0.026919, u_c=0.026919, diff=-7.450581e-09
t=18, i=7, u_py=0.014509, u_c=0.014509, diff=-9.313226e-10
t=19, i=0, u_py=0.324196, u_c=0.324196, diff=-8.940697e-08
t=19, i=1, u_py=0.024471, u_c=0.024471, diff=-1.862645e-09
t=19, i=2, u_py=0.009621, u_c=0.009621, diff=-9.313226e-10
t=19, i=3, u_py=0.018169, u_c=0.018169, diff=3.725290e-09
t=19, i=4, u_py=0.019826, u_c=0.019826, diff=1.862645e-09
t=19, i=5, u_py=0.315268, u_c=0.315268, diff=-2.384186e-07
t=19, i=6, u_py=0.026964, u_c=0.026964, diff=-3.725290e-09
t=19, i=7, u_py=0.014777, u_c=0.014777, diff=0.000000e+00
t=3, i=0, s_py=1, s_c=1, match=True
t=7, i=5, s_py=1, s_c=1, match=True
t=9, i=0, s_py=1, s_c=1, match=True
(venv) enre@emre-ABRA-A5-V16-6:~/AllProjects/GPU-Accelerated-LIF-Spiking-N(venv)

```

Figure 2: Comparison output of C and pytorch versions

5 Testing

5.1 Method

Example Code

Figure 3: Transformation function definitons

Figure 4: Pareto graph of different image transformation techniques

Table 1: Power saving statistics for each transformation

Transform	Mean PS (%)	Min PS (%)	Max PS (%)
Bright scale	17.18	16.85	18.46
Histogram	-5.53	-135.09	30.56
Hungry blue	6.43	2.78	20.24

References

- [1] Matei-Ioan Stan and Oliver Rhodes. Learning long sequences in spiking neural networks. *Scientific Reports*, 14(1):21774, 2024.